

5 Databáze (specializace DW)

Je dán následující transakční rozvrh S :

T_1	T_2	T_3
$R(A)$	$W(B)$	$W(A)$
$W(B)$	$R(A)$	$W(B)$
COMMIT	COMMIT	COMMIT

1. Je rozvrh S konfliktově uspořádatelný (conflict-serializable)? Své rozhodnutí vysvětlete.
2. Je rozvrh S zotavitelný? Své rozhodnutí vysvětlete. Pokud ne, bylo by možné zotavitelnosti dosáhnout bez toho, že by se změnilo současné vzájemné pořadí operací R a W ? Jak?

Do rozvrhu S byly doplněny operace pro uzamykání L_X (exkluzivní zámek) a L_S (sdílený zámek) s cílem získat rozvrh odpovídající požadavkům pro striktní dvoufázový uzamykací protokol (strict 2PL).

T_1	T_2	T_3
$L_S(A)$ $R(A)$	$L_X(B)$ $W(B)$	$L_X(A)$ $W(A)$
$L_X(B)$ $W(B)$	$L_S(A)$ $R(A)$	$L_X(B)$ $W(B)$
COMMIT	COMMIT	COMMIT

3. Bylo cíle dosaženo? Své rozhodnutí vysvětlete.

Nástin řešení

1. Ano, je. Precedenční graf obsahuje hrany $T_1 \rightarrow T_1$, $T_2 \rightarrow T_1$, $T_2 \rightarrow T_3$ a $T_1 \rightarrow T_3$. V grafu není orientovaný cyklus. Je tedy topologicky uspořádatelný.
2. Ne, není. T_2 čte A , zapsané v T_3 , ale dělá *COMMIT* dříve než T_3 . Pro opravu musí být *COMMIT* v T_3 pozdržen a proveden až po *COMMIT* v T_3 .
3. Ne, nebylo. Obecně to nejde, protože rozvrh, který odpovídá strict 2PL protokolu, je jak konfliktově uspořádatelný, tak zotavitelný. Doplnění vytvoří nekorektní rozvrh. Zde m.j. $L_X(A)$ v T_3 tuto transakci zablokuje do *COMMIT* v T_1 .

6 Datový management (specializace DW)

Do netříděného sekvenčního souboru s délkou bloku 4096 B je uloženo 8192 záznamů, každý s pevnou délkou 500 B.

1. Kolik pater bude mít hierarchický index nad sloupcem reprezentujícím klíč jednotlivých záznamů, pokud se do jednoho bloku indexu vejde 64 položek? Jedná se o přímý nebo nepřímý index? Svoje odpovědi zdůvodněte.
2. Jak se odpověď změní (pokud vůbec) v případě, že bude primární soubor uložen jako tříděný sekvenční soubor seřazený podle hodnot klíče? Svě rozhodnutí vysvětlíte.
3. Popište vkládání hodnot do neredundandního B-stromu s řádem m . Znázorněte, jak bude vypadat neredundandní B-strom ($m=3$) po vložení prvků 9, 3, 7, 6, 5 v tomto pořadí.

Nástin řešení

1. Bude potřeba adresovat všech 8192 položek primárního souboru v první úrovni, což zabere 128 bloků. 128 položek druhé úrovně zabere 2 bloky. 2 položky ve třetí úrovni se vejdou do jednoho kořenového bloku. Jedná se o přímý (odkazy adresují přímo primární soubor) index.
2. Ano, změní. Bude potřeba adresovat jen 1024 bloků v indexu první úrovně, což zabere 16 bloků. 16 položek se vejde do jednoho bloku druhé úrovně. Stále se jedná o přímý (odkazy adresují přímo primární soubor) index.
3. V případě vkládání 3. prvku do plného uzlu se dvěma prvky se v uzlu ponechají krajní hodnoty a prostřední se vloží o patro výše. Strom tedy bude vypadat:

```
          (-, -)
i 9      (9, -)
i 3      (3, 9)
i 7      (7, -)
      (3, -)      (9, -)
i 6      (7, -)
      (3, 6)      (9, -)
i 5      (5, 7)
      (3, -) (6, -) (9, -)
```

7 Web app: vyhledávání článků (specializace DW)

Mějme webovou aplikaci pro vyhledávání článků. Uživatel má k dispozici textový vstup (input element s ID `searchInput`) a vyhledávací tlačítko (button element s ID `searchButton`). Při stisku tlačítka se uživateli zobrazí všechny články, které obsahují text ze vstupního pole jako podřetězec.

Aplikace používá REST API, z něhož potřebujete dva endpointy:

`/api/articles?search=<text-fragment>` vrací články, které obsahují daný řetězec, jako JSON seznam:

```
[ { "id": <id článku>, "content": <html fragment s obsahem>, "author": <id autora>, ... }, ... ]
```

`/api/user/<id>` vrací údaje o jednom uživateli jako JSON kolekci:

```
{ "id": <id uživatele>, "name": <celé jméno>, ... }
```

Pro jednoduchost již máte připravené funkce pro úpravu DOM: `clearArticles()` odstraní aktuálně zobrazované články a `addArticle(content, author)` přidá na stránku nový článek od daného autora.

1. Napište fragment JavaScriptu, který zajistí popisovanou funkcionalitu. Tedy při stisku tlačítka se provedou HTTP požadavky na příslušné endpointy z REST API a odpovídajícím způsobem se upraví DOM model (zobrazí se stažené články). Ve vašem řešení máte především demonstrovat práci s asynchronními požadavky (pomocí promises nebo `async/await`). Rovněž je nutné zajistit, aby se opakované stisky tlačítka během načítání článků ignorovaly.
2. Naznačte, jak by mohla vypadat implementace funkce `addArticle`. Není třeba psát celý kód, důležité je především nastínit, jak se do stránky vloží HTML fragment z položky `content` zaslané z REST API.

Ve vašem kódu nemusíte řešit okrajové situace (selhání sítě, chyby v datech, apod.). Pokud si nejste jisti přesnými názvy standardních funkcí a metod JS nebo DOM API, doplňte do komentáře, co očekáváte, že daná funkce dělá. Drobné chyby v syntax budou tolerovány.

Nástin řešení

1. Příklad řešení s použitím `async/await`. Hlavními očekávanými body řešení jsou správné použití `async/await` (nebo promises) a pak lock guard, naparsování JSONu a správné navázání event handleru.

```
var lock = false; // guard, který zajistí, aby tlačítko nefungovalo, když probíhá async operace
document.getElementById('searchButton').addEventListener('click', async () => {
  const query = document.getElementById('searchInput').value();
  if (query && !lock) {
    lock = true;
    clearArticles();
    const articlesResponse = await fetch(`/api/articles?search=${encodeURIComponent(query)}`);
    const articles = await articlesResponse.json(); // přesně parsování JSON není důležité
    for (const article of articles) {
      const authorResponse = await fetch(`/api/user/${encodeURIComponent(article.author)}`);
      const author = await authorResponse.json();
      addArticle(article.content, author.name);
    }
    lock = false;
  }
});
```

2. Stačí vyrobit element `<article>` a do property `innerHTML` mu přiřadit řetězec `content`. Alternativa je použít `DOMParser`, ale to je zbytečně složité. Přesný název není důležitý, ale je potřeba ukázat, že nastavit HTML fragment jako text nestačí (pak se to skutečně vloží jako text element, bez ohledu na to, že ve stringu jsou tagy).

8 Bezpečnostní tokeny (specializace DW)

Webová aplikace používá bezpečnostní tokeny JWT pro interní udržování autentizace (tedy autentizaci HTTP dotazů po té, co se uživatel přihlásil loginem a heslem). Token má hlavičku

```
{ "alg": "HS256", "typ": "JWT" }
```

kde HS256 je identifikátor metody *Hash-Based Message Authentication Code* (HMAC) s použitím SHA256 jako hašovací funkce (pro připomenutí: HS256 je výchozí metoda, kterou musí podporovat všechny implementace JWT knihoven, a se kterou se potkáme nejčastěji).

1. Stručně popište (nejlépe algoritmicky pomocí funkcí) jak je zajištěna bezpečnost tohoto typu tokenu — tedy jak probíhá **vydání** a **ověření** tokenu bezpečnostní entitou (serverem).
2. Navrhnete, jaké položky musí mít tělo (payload) JWT tokenu, aby plnil autentizační funkci a byly zajištěny základní zásady bezpečnosti. Váš návrh by neměl obsahovat zbytečné položky (tokeny mají být malé). Jednotlivé položky stručně (jednou větou) vysvětlte/zdůvodněte.
3. Popište dva nejčastější mechanismy pro perzistentní uchování tokenu na straně klienta (tj. v prohlížeči) a stručně porovnejte jejich výhody nevýhody.

Nástin řešení

1. Viz <https://jwt.io/introduction>. Token se skládá ze 3 částí – Header, Payload, Signature – zakódovaných pomocí Base64. Server má tajný klíč (Secret), kterým tokeny podepisuje a ověřuje. Přičemž $\text{Signature} = \text{SHA256}(\text{Header} + \text{Payload} + \text{Secret})$. Tj. při vydávání tokenu se správně spočítá Signature, která je díky Secret neduplikovatelná, při ověření se úplně stejným způsobem spočítá znovu a porovná se Signature uloženou v tokenu.
2. Minimální payload obsahuje:
 - Někjaký identifikátor uživatele (ideálně databázové ID, neměl by to být osobní identifikátor jako RČ nebo email, ani login, už vůbec by to nemělo být něco, co je možné měnit).
 - Čas expirace, jinak by token platil navždy, což není žádoucí. Alternativně je možné místo exp uvádět iat (čas vydání), nebo dokonce oboje (v některých scénářích se to může hodit).

Rozumné další položky jsou třeba role (pokud jich uživatel může mít více) nebo omezení autorizačního scope (např. read-only tokeny).

3. Existují právě dva rozumné a všeobecně podporované mechanismy – cookies a local storage (alternativně session storage). Hlavní rozdíl je, že cookies se spravují automaticky prohlížečem (může je rovnou nastavit server přes HTTP hlavičky) a automaticky se posílají s každým požadavkem (nemusíme se o ně starat). Cookies jsou také jediná cesta, pokud potřebujeme podporu SSR pro SPA aplikace. Token v local storage se musí přidávat do hlaviček každého async. requestu (HTTP hlavička **Authorization: Bearer <token>**), ale zase nad tím má programátor plnou kontrolu (hodí se např. pokud aplikace spravuje více tokenů pro přepínání rolí).